AJAY.I - 71812401007 · FOUNDATIONS OF AGENTIC AI · REVISION CODES

Experiment 1 — Implement the OODA loop

exp1.py

```
def observe():
    print("\n--- OBSERVE ---")
    temp = float(input("Enter temperature (C): "))
    rain = input("Is it raining? (yes/no): ").lower() == "yes"
    hours = float(input("Enter free hours: "))
    return temp, rain, hours

def orient(temp, rain, hours):
    outdoor = not rain and temp <= 35
    long_study = hours >= 3
    print("--- ORIENT ---")
    print("Outdoor suitable:", outdoor)
    print("Long study possible:", long_study)
    return outdoor, long_study

def decide(outdoor, study):
    print("--- DECIDE ---")
    if outdoor and study:
        return "Study in library, then take a walk"
    elif outdoor:
        return "Take a short walk and revise notes"
    elif study:
        return "Stay indoors and study for three hours"
    else:
        return "Stay indoors and revise for one hour"

def act(decision):
    print("--- ACT ---")
    print("Executing:", decision)

print("=== OODA Study Assistant ===")
for cycle in range(1, 3):
    print("\nCycle", cycle)
    temp, rain, hours = observe()
    outdoor, study = orient(temp, rain, hours)
    decision = decide(outdoor, study)
    print("Decision:", decision)
    act(decision)

print("\nOODA loop completed.")
```

Experiment 2 — LLM-based agent using the Thought–Action cycle

exp2.py

```
import re

def think(task):
    text = task.lower()
    nums = [float(n) for n in re.findall(r"\d+\.?\d*", text)]
    if "add" in text and len(nums) >= 2:
        return "Addition requested", "add", nums
    if "multiply" in text and len(nums) >= 2:
        return "Multiplication requested", "multiply", nums
    if "reverse" in text:
        return "Reverse text", "reverse", [text.split("reverse",
1)[1].strip()]
    if "count" in text and "words" in text:
        return "Count words", "count", [task.split(":", 1)
[-1].strip()]
    return "No matching action", "none", []

def act(action, args):
    if action == "add":
        return sum(args)
    if action == "multiply":
        result = 1
        for n in args:
            result *= n
        return result
    if action == "reverse":
        return args[0][::-1]
    if action == "count":
        return len(args[0].split())
    return "Unable to complete task"

print("=== Thought-Action Agent ===")
print("Rule-based simulation; no LLM API used.")

while True:
    task = input("\nEnter task (or exit): ").strip()
    if task.lower() == "exit":
        print("Agent stopped.")
        break
    thought, action, args = think(task)
    print("Thought:", thought)
    print("Action:", action, args)
    print("Result:", act(action, args))
```

Experiment 3 — Develop an agent using the Observation cycle

exp3.py

```
readings = [24, 28, 33, 38, 31, 22]
fan = "OFF"

print("=== Temperature Monitoring Agent ===")

for i, temp in enumerate(readings, 1):
    print(f"Cycle {i}: Temperature = {temp} C")

    if temp < 26:
        state = "Cool"
    elif temp < 32:
        state = "Warm"
    else:
        state = "Hot"

    if state == "Hot":
        action = "Turn fan ON at HIGH speed"
        fan = "HIGH"
    elif state == "Warm" and fan == "OFF":
        action = "Turn fan ON at LOW speed"
        fan = "LOW"
    elif state == "Cool" and fan != "OFF":
        action = "Turn fan OFF"
        fan = "OFF"
    else:
        action = "No change"

    print("Interpretation:", state)
    print("Action:", action)
    print("Fan state:", fan)
    print()

print("Monitoring completed.")
```

Experiment 4 — Implement tool calling

exp4.py

```
import re

def calculator(a, b, op):
    if op == "+": return a + b
    if op == "-": return a - b
    if op == "*": return a * b
    if op == "/": return a / b if b != 0 else "Division by zero"

def convert(value, unit):
    if unit == "km":
        return f"{value * 0.621371:.2f} miles"
    if unit == "kg":
        return f"{value * 2.20462:.2f} pounds"
    if unit == "c":
        return f"{value * 9 / 5 + 32:.1f} F"

def weather(city):
    data = {
        "chennai": "32 C, humid",
        "coimbatore": "27 C, cloudy"
    }
    return data.get(city, "No data available")

print("=== Tool Calling Agent ===")

while True:
    request = input("\nRequest (or exit): ").lower().strip()
    if request == "exit":
        print("Agent stopped.")
        break

    if "convert" in request:
        m = re.search(r"(\d+\.?\d*)\s*(km|kg|c)\b", request)
        print("Tool selected: unit converter")
        if m:
            print("Result:", convert(float(m.group(1)),
m.group(2)))
        else:
            print("Result: Invalid conversion request")
    elif "weather" in request:
        city = request.split()[-1]
        print("Tool selected: weather")
        print("Result:", weather(city))
    else:
        m = re.search(r"(\d+\.?\d*)\s*([+\-*/])\s*(\d+\.?\d*)",
request)
        if m:
            print("Tool selected: calculator")
            print("Result:", calculator(float(m.group(1)),
                                        float(m.group(3)),
m.group(2)))
        else:
            print("No suitable tool found")
```

Experiment 5 — Structured output generation

exp5.py

```
import json
import re

def generate(request):
    text = request.lower()
    priority = "medium"
    for level in ("low", "medium", "high"):
        if level in text:
            priority = level
    match = re.search(r"(\d+\.?\d*)\s*hours?", text)
    duration = float(match.group(1)) if match else None
    task = re.split(r",| for \d", request)[0].strip()
    return {
        "task": task,
        "priority": priority,
        "duration_hours": duration,
        "status": "pending"
    }

def validate(data):
    errors = []
    if not data["task"]:
        errors.append("Task cannot be empty")
    if data["priority"] not in ("low", "medium", "high"):
        errors.append("Invalid priority")
    if data["duration_hours"] is None or data["duration_hours"] <=
0:
        errors.append("Duration must be positive")
    return errors

print("=== Structured Output Generator ===")

while True:
    request = input("\nStudy request (or exit): ").strip()
    if request.lower() == "exit":
        print("Generator stopped.")
        break
    data = generate(request)
    errors = validate(data)
    if errors:
        print("Validation FAILED:", errors)
    else:
        print("Validation PASSED")
        print(json.dumps(data, indent=2))
```

Experiment 6 — Compare System 1 and System 2 reasoning

exp6.py

```
laptops = [
    {"name": "Laptop A", "price": 38000, "speed": 5, "battery":
6},
    {"name": "Laptop B", "price": 55000, "speed": 8, "battery":
9},
    {"name": "Laptop C", "price": 72000, "speed": 9, "battery":
7},
    {"name": "Laptop D", "price": 45000, "speed": 7, "battery": 5}
]

def system1(budget):
    affordable = [x for x in laptops if x["price"] <= budget]
    return min(affordable, key=lambda x: x["price"])

def score(x, budget):
    return (0.3 * ((budget - x["price"]) / budget * 10)
            + 0.4 * x["speed"] + 0.3 * x["battery"])

def system2(budget):
    affordable = [x for x in laptops if x["price"] <= budget]
    return max(affordable, key=lambda x:
        0.3 * ((budget - x["price"]) / budget * 10)
        + 0.4 * x["speed"] + 0.3 * x["battery"])

print("=== System 1 vs System 2 ===")

for budget in (40000, 60000, 80000):
    fast = system1(budget)
    deep = system2(budget)
    print("\nBudget: Rs.", budget)
    print("System 1:", fast["name"])
    print("System 2:", deep["name"], "Score:", round(score(deep,
budget), 2))
    print("Same decision:", fast["name"] == deep["name"])
```

Experiment 7 — CoALA-based cognitive architecture

exp7.py

```
class Environment:
    def __init__(self):
        self.temperature = 28
        self.ac_on = False

    def observe(self):
        return {"temperature": self.temperature, "ac_on":
self.ac_on}

    def execute(self, action):
        if action == "turn_on_ac":
            self.ac_on = True
        elif action == "turn_off_ac":
            self.ac_on = False
        self.temperature += -2 if self.ac_on else 3
        return self.temperature

class Agent:
    def __init__(self):
        self.working = {}
        self.episodic = []
        self.semantic = "Temperature above 30 needs cooling"

    def decide(self, obs):
        self.working = dict(obs)
        temp = obs["temperature"]
        if temp > 30:
            print("Knowledge:", self.semantic)
        if temp > 30 and not obs["ac_on"]:
            return "turn_on_ac"
        if temp < 24 and obs["ac_on"]:
            return "turn_off_ac"
        return "wait"

    def remember(self, obs, action, new_temp):
        self.episodic.append((obs, action, new_temp))

env = Environment()
agent = Agent()

print("=== CoALA Cognitive Agent ===")

for step in range(1, 10):
    obs = env.observe()
    print("Step:", step, "| Observation:", obs)
    action = agent.decide(obs)
    new_temp = env.execute(action)
    agent.remember(obs, action, new_temp)
    print("Action:", action, "| New temperature:", new_temp)
    print()

print("Episodic memory:", len(agent.episodic), "episodes")
```

Experiment 8 — Gymnasium-based intelligent agent environment

exp8.py

```
import gymnasium as gym
from gymnasium import spaces

class GridWorldEnv(gym.Env):
    def __init__(self):
        super().__init__()
        self.observation_space = spaces.Discrete(16)
        self.action_space = spaces.Discrete(4)
        self.state = 0
        self.steps = 0

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self.state = 0
        self.steps = 0
        return self.state, {}

    def step(self, action):
        row, col = divmod(self.state, 4)
        if action == 0:
            row = max(row - 1, 0)
        elif action == 1:
            row = min(row + 1, 3)
        elif action == 2:
            col = max(col - 1, 0)
        else:
            col = min(col + 1, 3)
        self.state = row * 4 + col
        self.steps += 1
        terminated = self.state == 15
        reward = 10.0 if terminated else -0.1
        return self.state, reward, terminated, False, {}

env = GridWorldEnv()
names = {0: "Up", 1: "Down", 2: "Left", 3: "Right"}
state, _ = env.reset()
print("Observation space:", env.observation_space)
print("Action space:", env.action_space)
print("Start state:", state)

total = 0
for action in [3, 3, 3, 1, 1, 1]:
    state, reward, terminated, truncated, _ = env.step(action)
    total += reward
    print("Action:", names[action], "State:", state,
          "Reward:", reward, "Terminated:", terminated)
    if terminated:
        break

print("Total reward:", round(total, 2))
```

Experiment 9 — ReAct, Reflection and Planning

exp9.py

```
scores = {
    "Revise notes": 85,
    "Practice programs": 55,
    "Mock test": 70
}

def plan(hours):
    tasks = [("Revise notes", 2),
             ("Practice programs", 3),
             ("Mock test", 2)]
    if sum(h for _, h in tasks) > hours:
        tasks = [(name, max(1, h - 1)) for name, h in tasks]
    print("PLANNING:", tasks)
    return tasks

def react(task, hours):
    print("\nTask:", task)
    print("Thought: I need", hours, "hours.")
    print("Action: Start", task)
    result = scores[task]
    print("Observation: Score =", result)
    return result

def reflect(task, score):
    if score < 60:
        print("Reflection: Low score. Revision needed.")
        return task
    print("Reflection: Score acceptable.")
    return None

print("=== ReAct, Reflection and Planning ===")
hours = int(input("Enter available hours: "))
revise = []

for task, h in plan(hours):
    score = react(task, h)
    weak = reflect(task, score)
    if weak:
        revise.append(weak)

print("\nREFLECTION SUMMARY")
if revise:
    print("Add revision for:", ", ".join(revise))
else:
    print("No extra revision needed")
print("Study plan completed.")
```

Experiment 10 — Memory-enabled agent using RAG and MemGPT concepts

exp10.py

```
import re
import sqlite3

knowledge = [
    ("RAG", "RAG retrieves documents and uses them to answer
questions."),
    ("MemGPT", "MemGPT-style agents use core and archival
memory."),
    ("Tool calling", "Tool calling selects a function and passes
arguments.")
]

db = sqlite3.connect("rag_memory.db")
db.execute("CREATE TABLE IF NOT EXISTS memory "
           "(key TEXT PRIMARY KEY, value TEXT)")
db.commit()

def remember(key, value):
    db.execute("INSERT OR REPLACE INTO memory VALUES (?, ?)",
               (key, value))
    db.commit()

def recall(key):
    row = db.execute("SELECT value FROM memory WHERE key=?",
                     (key,)).fetchone()
    return row[0] if row else None

def retrieve(question):
    q = question.lower()
    for title, text in knowledge:
        if title.lower() in q:
            return f"[Retrieved: {title}] {text}"
    words = set(re.findall(r"\w+", q))
    for title, text in knowledge:
        if words & set(re.findall(r"\w+", text.lower())) - {"and",
"the", "a"}:
            return f"[Retrieved: {title}] {text}"
    return "No relevant knowledge found."

print("=== RAG Agent with Memory ===")
print("Stored style:", recall("style"))
print("Stored topic:", recall("topic"))

while True:
    msg = input("\nYou: ").strip()
    low = msg.lower()
    if low == "exit":
        print("Session ended. Memory saved.")
        break
    if low.startswith("remember "):
        key, _, value = msg[9:].partition("=")
        remember(key.strip(), value.strip())
        print("Agent: Saved", key.strip(), "=", value.strip())
    elif low.startswith("what is my "):
        print("Agent:", recall(low[11:].strip()) or "I do not know
yet.")
    else:
        print("Agent:", retrieve(msg))
```